

Section 2: Neural Network Architecture Deep Dive

Table of Contents

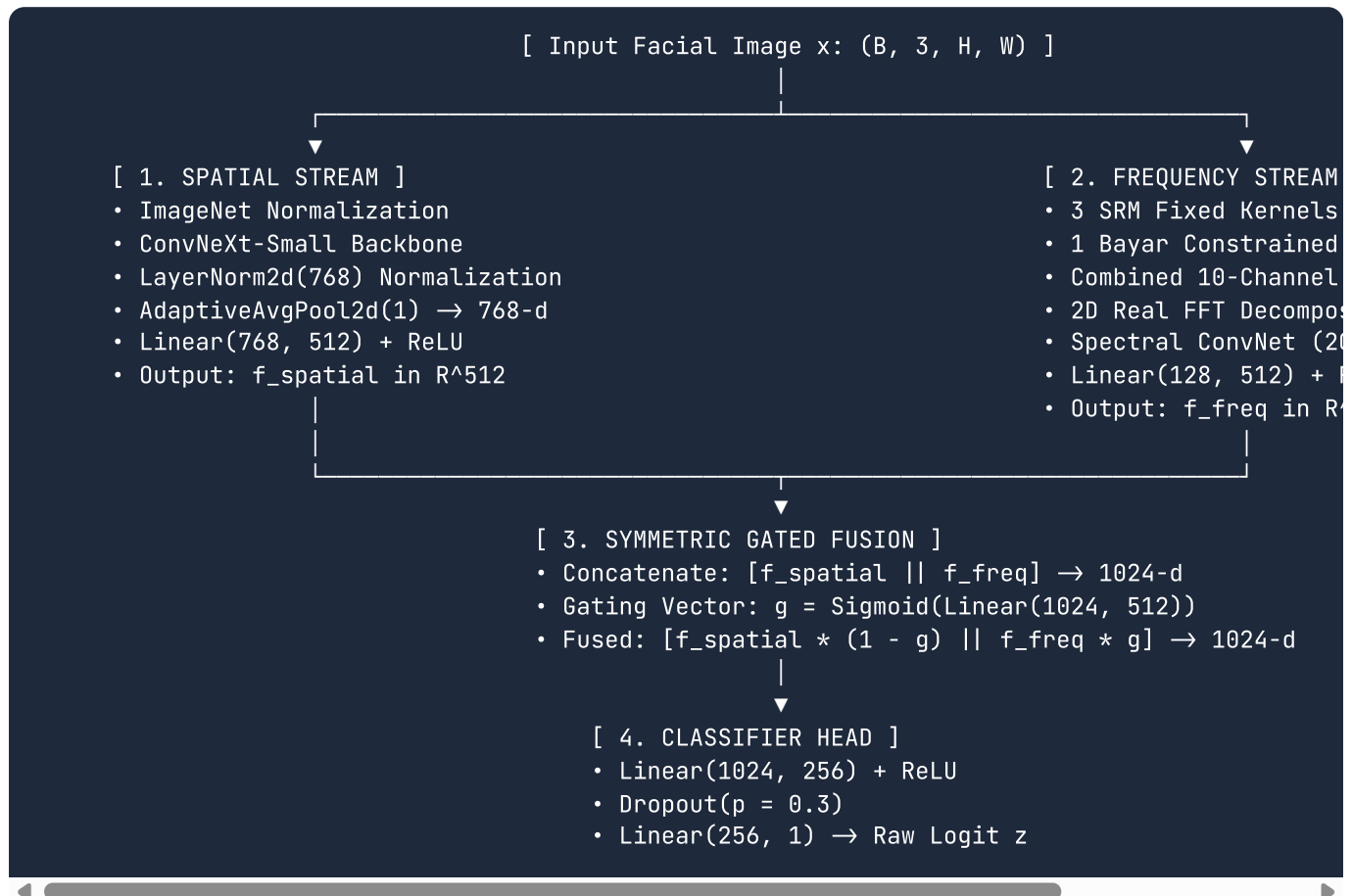
1. Overview & Architectural Philosophy
 2. PyTorch Fundamentals & Module Hierarchy
 3. The Spatial Stream: ConvNeXt-Small Backbone
 4. 3.1 Why ConvNeXt-Small? (Modern ConvNet Design)
 5. 3.2 Input Preprocessing & ImageNet Standardization
 6. 3.3 Feature Extraction Stages
 7. 3.4 The Critical Role of Pre-Classifier LayerNorm2d
 8. 3.5 Spatial Projection to 512 Dimensions
 9. The Frequency Stream: Steganographic & Spectral Decomposition
 10. 4.1 Spatial Rich Model Filter Bank (`SRMConv2d`)
 11. 4.2 Bayar-Stamm Constrained Convolution (`BayarConv2d`)
 12. 4.3 10-Channel Noise Residual Assembly
 13. 4.4 2D Real FFT Spectral Decomposition (`RealFFT2DModule`)
 14. 4.5 Why FP32 is Mandatory (Autocast Disabling)
 15. 4.6 Spectral Convolutional Reduction to 512 Dimensions
 16. Symmetric Gated Residual Fusion
 17. 5.1 Mathematical Motivation: Avoiding Gradient Starvation
 18. 5.2 Gating Vector Computation ($g \in \mathbb{R}^{512}$)
 19. 5.3 Symmetrical Feature Modulation (1024-d)
 20. The Binary Classification Head
 21. Temporal Sequence Processing (`forward_sequence`)
 22. 7.1 5D Video Tensor Ingestion
 23. 7.2 Chunked Inference to Prevent GPU Out-of-Memory (OOM)
 24. 7.3 Logit Averaging over Temporal Sequences
 25. Complete Forward Pass: Tensor Shape Lifecycle Trace
 26. Code Walkthrough & Reference
-

1. Overview & Architectural Philosophy

The entire model architecture is defined in `src/models/hybrid_detector.py` .

The detector operates on the principle that **facial manipulation creates distinct artifacts in two orthogonal domains**: 1. **Spatial Domain**: High-level semantic inconsistencies, facial boundary blending seams, unnatural skin texture, and lighting mismatches. 2. **Frequency Domain**: Microscopic noise discrepancies, steganographic residual disruptions, and repeating upsampling checkerboard grid patterns.

Rather than forcing a single network to learn both macroscopic semantics and microscopic noise simultaneously, the architecture decouples them into two specialized parallel streams and joins them via a **Symmetric Gated Residual Fusion** mechanism.



2. PyTorch Fundamentals & Module Hierarchy

In PyTorch, all neural network layers inherit from `torch.nn.Module`. A module encapsulates: 1. **Parameters (`nn.Parameter`)**: Learnable weights and biases updated via backpropagation gradients ($\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}$). 2. **Buffers (`register_buffer`)**: Persistent state tensors (such as fixed SRM filter weights or ImageNet running statistics) that are not updated by gradient descent, but must move to the GPU alongside the model and be saved in checkpoint state dictionaries. 3. **The `forward(*args)` Method**: The computational graph executed during inference and training.

The model hierarchy in `src/models/hybrid_detector.py` consists of 4 distinct modules: - `SRMConv2d`: Fixed high-pass steganographic filter bank. - `BayarConv2d`: Learnable constrained convolutional layer. - `RealFFT2DModule`: 2D Fast Fourier spectral decomposition module. - `HybridDeepfakeDetector`: The master dual-stream fusion container.

3. The Spatial Stream: ConvNeXt-Small Backbone

3.1 Why ConvNeXt-Small? (Modern ConvNet Design)

Historically, deepfake detectors relied on ResNet-50 or EfficientNet. ConvNeXt (Liu et al., CVPR 2022) modernized standard convolutional networks by incorporating architectural innovations from Vision Transformers (ViT) while retaining convolutional inductive bias:

- 7×7 Depthwise Separable Convolutions:** Matches the large receptive field of ViT self-attention windows.
- Inverted Bottleneck Design:** Expands hidden channel dimensions (1×1 conv $\rightarrow 7 \times 7$ depthwise $\rightarrow 1 \times 1$ projection) rather than contracting them.
- Fewer Normalization & Activation Layers:** Uses **GELU** (Gaussian Error Linear Unit) instead of ReLU, and LayerNorm instead of BatchNorm, resulting in smoother gradient landscapes.
- Parameter Efficiency:** `convnext_small` achieves superior feature representations with ~50 million parameters compared to larger ViT variants.

3.2 Input Preprocessing & ImageNet Standardization

In `HybridDeepfakeDetector.__init__`, ImageNet mean and standard deviation are registered as persistent buffers:

```
self.register_buffer("imagenet_mean", torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1))
self.register_buffer("imagenet_std", torch.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1))
```

During the spatial forward pass:

$$x_{\text{spatial}} = \frac{x - \mu_{\text{imagenet}}}{\sigma_{\text{imagenet}}}$$

3.3 Feature Extraction Stages

The input image $x_{\text{spatial}} \in \mathbb{R}^{B \times 3 \times H \times W}$ passes through `self.spatial_backbone` (`convnext_small.features`), which consists of 4 successive downsampling stages:

- Stage 1 (Stem):** 4×4 patchify conv with stride 4 $\Rightarrow \mathbb{R}^{B \times 96 \times \frac{H}{4} \times \frac{W}{4}}$
- Stage 2:** Downsample + ConvNeXt Blocks $\Rightarrow \mathbb{R}^{B \times 192 \times \frac{H}{8} \times \frac{W}{8}}$
- Stage 3:** Downsample + ConvNeXt Blocks $\Rightarrow \mathbb{R}^{B \times 384 \times \frac{H}{16} \times \frac{W}{16}}$
- Stage 4:** Downsample + ConvNeXt Blocks $\Rightarrow \mathbb{R}^{B \times 768 \times \frac{H}{32} \times \frac{W}{32}}$

For an input resolution of $H = W = 256$, the final spatial feature map has shape: `\text{feat\_maps}` in `\mathbb{R}^{B \times 768 \times 8 \times 8}`

3.4 The Critical Role of Pre-Classifier LayerNorm2d

In standard torchvision implementations, `convnext.classifier[0]` is an `nn.LayerNorm2d(768)` layer placed between the feature extractor and the classification head.

```
self.spatial_norm = convnext.classifier[0] # nn.LayerNorm2d(768)
```

Why this is mathematically essential: Without `self.spatial_norm`, the spatial feature activations have unconstrained variances ($\sigma^2 > 15.0$). When concatenated with the normalized frequency stream ($\sigma^2 \approx 1.0$), the unscaled spatial features dominate the gating mechanism, causing severe training instability and volatile gradients. Applying `spatial_norm` standardizes spatial activations to zero mean and unit variance before pooling.

3.5 Spatial Projection to 512 Dimensions

- Global Adaptive Average Pooling:** $f_{\text{pooled}} = \text{AdaptiveAvgPool2d}(1)(\text{feat_maps})$
 $\implies \mathbb{R}^{B \times 768 \times 1 \times 1} \xrightarrow{\text{flatten}} \mathbb{R}^{B \times 768}$
- Dense Linear Projection:** $\mathbf{f}_{\text{spatial}} = \text{ReLU}\big(\mathbf{W}_s \mathbf{f}_{\text{pooled}} + \mathbf{b}_s\big) \in \mathbb{R}^{B \times 512}$ where $\mathbf{W}_s \in \mathbb{R}^{512 \times 768}$.

4. The Frequency Stream: Steganographic & Spectral Decomposition

The frequency stream isolates high-frequency steganographic noise residuals, converts them into 2D Fourier space, and extracts a 512-dimensional spectral representation.

4.1 Spatial Rich Model Filter Bank (`SRMConv2d`)

Implemented in `SRMConv2d`.

Three 5×5 high-pass filter kernels are defined in NumPy:

```
# 1st-Order High-Pass Difference Kernel (divided by 4.0)
srm1 = np.array([
    [ 0,  0,  0,  0,  0],
    [ 0, -1,  2, -1,  0],
    [ 0,  2, -4,  2,  0],
    [ 0, -1,  2, -1,  0],
    [ 0,  0,  0,  0,  0]
], dtype=np.float32) / 4.0

# 2nd-Order High-Pass Laplacian Kernel (divided by 12.0)
srm2 = np.array([
    [-1,  2, -2,  2, -1],
    [ 2, -6,  8, -6,  2],
    [-2,  8, -12,  8, -2],
    [ 2, -6,  8, -6,  2],
    [-1,  2, -2,  2, -1]
], dtype=np.float32) / 12.0

# Horizontal/Vertical Edge Kernel (divided by 2.0)
srm3 = np.array([
    [ 0,  0,  0,  0,  0],
```

```

[ 0, 0, 0, 0, 0],
[ 0, 1, -2, 1, 0],
[ 0, 0, 0, 0, 0],
[ 0, 0, 0, 0, 0]
], dtype=np.float32) / 2.0

```

- The 3 kernels are stacked and tiled across the 3 RGB color channels to create a tensor of shape `[9, 1, 5, 5]`.
- Stored as a non-trainable parameter buffer: `self.register_buffer("weights", torch.from_numpy(filters))`.
- Executed via grouped 2D convolution (`groups=3`, `padding=2`):

$$I_{\text{SRM}} = \text{F.conv2d}(x, \mathbf{W}_{\text{SRM}}, \text{groups} = 3, \text{padding} = 2) \in \mathbb{R}^{B \times 9 \times H \times W}$$

- Channels 0, 1, 2: 3 SRM filters applied to Red channel.
- Channels 3, 4, 5: 3 SRM filters applied to Green channel.
- Channels 6, 7, 8: 3 SRM filters applied to Blue channel.

4.2 Bayar-Stamm Constrained Convolution (BayarConv2d)

Implemented in `BayarConv2d`.

Unlike the fixed SRM filters, the Bayar-Stamm layer contains a learnable kernel parameter:

$$\mathbf{W}_{\text{raw}} \in \mathbb{R}^{1 \times 3 \times 5 \times 5}$$

Before every convolution forward pass, the kernel is dynamically transformed by `_get_constrained_kernel()` to enforce the prediction-error constraint:

```

def _get_constrained_kernel(self) → torch.Tensor:
    w = self.kernel # [1, 3, 5, 5]
    mask = torch.ones_like(w)
    mask[:, :, 2, 2] = 0.0 # Zero out center coefficient (row 2, col 2)

    w_masked = w * mask
    sum_w = w_masked.sum(dim=(2, 3), keepdim=True)

    # Prevent division by zero with safe clamping
    sign_w = torch.sign(sum_w)
    sign_w = torch.where(sign_w == 0, torch.ones_like(sign_w), sign_w)
    sum_w_safe = sign_w * sum_w.abs().clamp(min=1e-5)

    # Normalize non-center weights so their sum equals exactly 1.0
    w_norm = w_masked / sum_w_safe

    # Set center coefficient to exactly -1.0
    center_mask = torch.zeros_like(w)
    center_mask[:, :, 2, 2] = -1.0
    return w_norm + center_mask

```

The constrained convolution yields 1 adaptive residual channel:

$$I_{\text{Bayar}} = \text{F.conv2d}(x, \mathbf{W}_{\text{constrained}}, \text{padding} = 2) \in \mathbb{R}^{B \times 1 \times H \times W}$$

4.3 10-Channel Noise Residual Assembly

The fixed SRM residuals and adaptive Bayar residuals are concatenated along the channel dimension:

$$I_{\text{noise}} = [I_{\text{SRM}} \mid I_{\text{Bayar}}] \in \mathbb{R}^{B \times 10 \times H \times W}$$

4.4 2D Real FFT Spectral Decomposition (`RealFFT2DModule`)

Implemented in `RealFFT2DModule` .

The 10-channel spatial noise residual tensor I_{noise} is transformed into frequency space:

```
# 1. 2D Orthogonal Fast Fourier Transform across spatial dimensions (H, W)
fft = torch.fft.fft2(x_fp32, norm="ortho")

# 2. Quadrant Shift to place zero-frequency (DC) at center
fft_shift = torch.fft.fftshift(fft, dim=(-2, -1))

# 3. Log-Magnitude Compression (log(1 + |FFT|))
mag = torch.log1p(torch.clamp(torch.abs(fft_shift), min=1e-7))

# 4. Normalized Phase Angle (atan2(Im, Re) / pi)
phase = torch.angle(fft_shift) / torch.pi

# 5. Numerical NaN/Inf Sanitization
mag = torch.nan_to_num(mag, nan=0.0, posinf=10.0, neginf=-10.0)
phase = torch.nan_to_num(phase, nan=0.0, posinf=1.0, neginf=-1.0)

# 6. Concatenate along channel dimension
out_spectral = torch.cat([mag, phase], dim=1) # [B, 20, H, W]
```

- **10 Magnitude Channels:** Captures periodic energy spikes and checkerboard grid frequencies.
- **10 Phase Channels:** Captures phase shifts and spatial coherence boundaries.
- **Total Spectral Channels:** $10 + 10 = 20$ channels of size $H \times W$.

4.5 Why FP32 is Mandatory (Autocast Disabling)

During PyTorch mixed-precision training (`torch.amp.autocast(device_type='cuda', dtype=torch.float16)`), general tensor operations execute in 16-bit float (`fp16`). However, `fp16` has an exponent range limited to $\approx 65,504$ and a smallest positive value of $\approx 6 \times 10^{-5}$. - Complex FFT operations compute exponential sums $\sum e^{-j2\pi(\dots)}$ that frequently exceed the dynamic range of `fp16`, causing immediate numerical underflow or `NaN` outputs. - In `RealFFT2DModule.forward`, autocasting is explicitly disabled during the spectral pass: `python with torch.amp.autocast(device_type=device_type, enabled=False): x_fp32 = x.float() # Execute FFT in full 32-bit floating point precision` The resulting tensor is then cast back to the model's active precision dtype.

4.6 Spectral Convolutional Reduction to 512 Dimensions

The $B \times 20 \times H \times W$ spectral map passes through `self.freq_conv` :

```
self.freq_conv = nn.Sequential(
    nn.Conv2d(20, 64, kernel_size=3, padding=1),
    nn.BatchNorm2d(64),
    nn.ReLU(),
    nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
    nn.BatchNorm2d(128),
    nn.ReLU(),
    nn.AdaptiveAvgPool2d(1),
)
```

1. Layer 1: $20 \rightarrow 64$ channels with spatial resolution preserved ($H \times W$).
2. Layer 2: $64 \rightarrow 128$ channels with stride 2 ($\frac{H}{2} \times \frac{W}{2}$).
3. Adaptive Global Pooling: Collapses spatial dimensions to $1 \times 1 \Rightarrow \mathbb{R}^{B \times 128}$.
4. Linear Projection: $\mathbf{f}_{\text{freq}} = \text{ReLU}(\text{big}(\mathbf{W}_f \mathbf{f}_{\text{freq_pooled}}) + \mathbf{b}_f)$ in $\mathbb{R}^{B \times 512}$ where $\mathbf{W}_f$ in $\mathbb{R}^{512 \times 128}$.

5. Symmetric Gated Residual Fusion

Implemented in `HybridDeepfakeDetector.forward` .

5.1 Mathematical Motivation: Avoiding Gradient Starvation

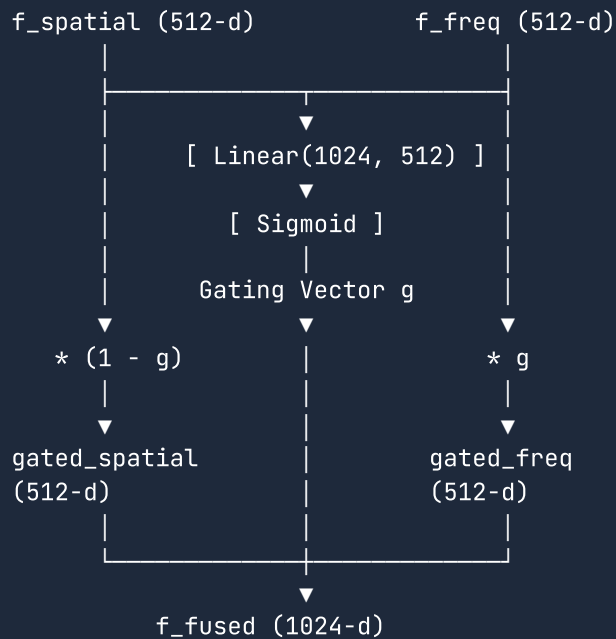
Consider an asymmetric gating model: $\mathbf{f}_{\text{fused}} = \text{big}(\mathbf{f}_{\text{spatial}} \parallel \mathbf{f}_{\text{freq}}) \odot g$ In this asymmetric setup, $\mathbf{f}_{\text{spatial}}$ has a direct, unimpeded gradient path: $\frac{\partial \mathcal{L}}{\partial \mathbf{f}_{\text{spatial}}} = \frac{\partial \mathcal{L}}{\partial \mathbf{f}_{\text{fused}}}[0:512]$ Because the spatial ConvNeXt backbone starts with pre-trained ImageNet weights, the optimizer can rapidly minimize training loss using spatial features alone. The network quickly learns to set $g \rightarrow 0$, permanently starving the frequency branch of gradient flow $\left(\frac{\partial \mathcal{L}}{\partial \mathbf{f}_{\text{freq}}} \rightarrow 0\right)$.

5.2 Gating Vector Computation ($g \in \mathbb{R}^{512}$)

To solve gradient starvation, we concatenate the 512-d spatial embedding and 512-d frequency embedding into a 1024-d joint feature vector: $\mathbf{f}_{\text{concat}} = \text{big}(\mathbf{f}_{\text{spatial}} \parallel \mathbf{f}_{\text{freq}})$ in $\mathbb{R}^{B \times 1024}$ The gating vector g is computed via a learned linear projection followed by a Sigmoid activation: $g = \text{sigma}(\text{big}(\mathbf{W}_g \mathbf{f}_{\text{concat}}) + \mathbf{b}_g)$ in $[0, 1]^{B \times 512}$ where $\mathbf{W}_g$ in $\mathbb{R}^{512 \times 1024}$.

5.3 Symmetrical Feature Modulation (1024-d)

The gating vector g simultaneously modulates **both** branches: $\mathbf{f}_{\text{fused}} = \text{Big}(\mathbf{f}_{\text{spatial}} \odot (1 - g) \parallel \mathbf{f}_{\text{freq}} \odot g)$ in $\mathbb{R}^{B \times 1024}$



Why this guarantees dual-stream optimization: 1. If $g_i \rightarrow 0$: The frequency stream is suppressed, but the spatial stream receives full weight ($1 - g_i \rightarrow 1$). 2. If $g_i \rightarrow 1$: The spatial stream is suppressed, and the frequency stream receives full weight. 3. If the classifier requires both spatial layout and spectral noise, $g_i \approx 0.5$, allowing gradients to flow equally back into both the ConvNeXt backbone and the SRM/Bayar/FFT convolutions.

6. The Binary Classification Head

Implemented in `HybridDeepfakeDetector.__init__`.

The 1024-dimensional fused representation $\mathbf{f}_{\text{fused}}$ passes through an MLP classifier:

```

self.classifier = nn.Sequential(
    nn.Linear(1024, 256),
    nn.ReLU(),
    nn.Dropout(p=0.3),
    nn.Linear(256, 1)
)

```

- First Dense Layer:** Projects $1024 \rightarrow 256$ dimensions with ReLU non-linearity.
- Dropout ($p = 0.3$):** Randomly zeros out 30% of activations during training to prevent co-adaptation and overfitting.
- Final Linear Layer:** Projects $256 \rightarrow 1$ scalar logit:
$$z = \mathbf{W}_2 \big(\text{Dropout}(\text{ReLU}(\mathbf{W}_1 \mathbf{f}_{\text{fused}} + \mathbf{b}_1)) + \mathbf{b}_2 \big) \in \mathbb{R}^{1 \times B}$$
- $z > 0 \implies p = \sigma(z) > 0.5$ (Predicted Fake)
- $z < 0 \implies p = \sigma(z) < 0.5$ (Predicted Real)

7. Temporal Sequence Processing (`forward_sequence`)

Implemented in `HybridDeepfakeDetector.forward_sequence` .

7.1 5D Video Tensor Ingestion

When performing inference or training on video sequences, inputs arrive as a **5D Tensor**:

$$x \in \mathbb{R}^{B \times T \times 3 \times H \times W}$$

where B is the batch size (number of videos) and T is the temporal sequence length (e.g., $T = 16$ sampled frames per video).

7.2 Chunked Inference to Prevent GPU Out-of-Memory (OOM)

If $B = 8$ and $T = 16$, flattening the tensor produces:

$$B \cdot T = 8 \times 16 = 128 \text{ full-resolution images}$$

Passing 128 high-resolution face crops through the dual-stream network simultaneously would exceed typical GPU VRAM (e.g., 16 GB on an NVIDIA T4).

`forward_sequence` prevents OOM crashes by processing frames in configurable sub-chunks (default `chunk_size = 8`):

```
batch_size, seq_len, c, h, w = x.shape
x_flat = x.view(batch_size * seq_len, c, h, w) # [B*T, 3, H, W]

logit_chunks: list[torch.Tensor] = []
for start in range(0, batch_size * seq_len, chunk_size):
    chunk = x_flat[start : start + chunk_size]
    logit_chunks.append(self.forward(chunk)) # [chunk_size, 1]

frame_logits = torch.cat(logit_chunks, dim=0) # [B*T, 1]
```

7.3 Logit Averaging over Temporal Sequences

The per-frame logits are reshaped back to the sequence dimension $[B, T]$ and averaged across time:

```
frame_logits = frame_logits.view(batch_size, seq_len) # [B, T]
video_logits = frame_logits.mean(dim=1, keepdim=True) # [B, 1]
return video_logits
```

This produces a single, stabilized video-level logit per sequence.

8. Complete Forward Pass: Tensor Shape Lifecycle Trace

The table below traces the exact shape transformations of a tensor throughout the model forward pass for a batch of $B = 4$ face crops of resolution 256×256 :

Network Stage / Operation	Output Tensor Shape	Description
0. Raw Input Image Batch	[4, 3, 256, 256]	RGB Face Crops in [0, 1]
SPATIAL STREAM PATHWAY		
1. ImageNet Standardization	[4, 3, 256, 256]	(x - mean) / std
2. ConvNeXt Feature Extraction	[4, 768, 8, 8]	Stage 4 backbone features
3. LayerNorm2d Normalization	[4, 768, 8, 8]	Standardized spatial features
4. Adaptive Global Average Pooling	[4, 768]	Collapsed spatial dimensions
5. Linear(768, 512) + ReLU	[4, 512]	Spatial Feature Vector
FREQUENCY STREAM PATHWAY		
6. SRMConv2d (3 filters x 3 channels)	[4, 9, 256, 256]	Fixed steganographic filters
7. BayarConv2d (Constrained Conv)	[4, 1, 256, 256]	Adaptive prediction of noise
8. Channel Concatenation	[4, 10, 256, 256]	Assembled noise residuals
9. 2D Real FFT + fftshift	[4, 10, 256, 256] (complex64)	Frequency domain representation
10. Log-Magnitude + Normalized Phase	[4, 20, 256, 256]	10 Mag + 10 Phase spectral components
11. Spectral Conv1 (20 → 64, pad 1)	[4, 64, 256, 256]	Spatial spectral feature extraction
12. Spectral Conv2 (64 → 128, str 2)	[4, 128, 128, 128]	Downsampled spectral features
13. Adaptive Global Average Pooling	[4, 128]	Collapsed spectral dimensions
14. Linear(128, 512) + ReLU	[4, 512]	Frequency Feature Vector
SYMMETRIC GATED FUSION & CLASSIFICATION		
15. Feature Concatenation	[4, 1024]	[f_spatial f_frequency]
16. Linear(1024, 512) + Sigmoid	[4, 512]	Gating Vector (g)
17. Symmetrical Gated Multiplication	[4, 1024]	[f_spatial*(1-g) f_frequency*g]
18. Linear(1024, 256) + ReLU	[4, 256]	Classifier hidden representation
19. Dropout(p=0.3)	[4, 256]	Regularized hidden representation
20. Linear(256, 1)	[4, 1]	Final Raw Logit Output

9. Code Walkthrough & Reference

The complete code implementation is organized in `src/models/hybrid_detector.py` :

Class / Function	Line Numbers	Purpose
<code>SRMConv2d</code>	Lines 20–50	Constructs the 9-channel fixed SRM steganographic high-pass filter bank.
<code>BayarConv2d</code>	Lines 51–80	Implements adaptive constrained convolution with automatic kernel normalization.
<code>RealFFT2DModule</code>	Lines 82–102	Executes 2D Real FFT decomposition, quadrant centering, and log-magnitude/phase extraction.
<code>HybridDeepfakeDetector.__init__</code>	Lines 111–166	Instantiates ConvNeXt spatial backbone, frequency branch, gating layer, and classifier head.
<code>HybridDeepfakeDetector.forward</code>	Lines 167–199	Executes full forward pass on 4D batch tensors $[B, 3, H, W]$.
<code>HybridDeepfakeDetector.forward_sequence</code>	Lines 200–223	Executes chunked forward pass on 5D video sequence tensors $[B, T, 3, H, W]$.

This document serves as the permanent reference for Section 2 (Neural Network Architecture) of the Deepfake Detection Engine.